EX12 - Movimiento vertical con funciones - Descomponer responsabilidades

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex12_vertical_motion_functions.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Reorganizar EX08 en funciones separadas para validar, simular, ejecutar un paso Euler, calcular referencia analitica y errores.

Variables / elementos del problema

Nombre	Significado
validate_inputs	valida entradas
simulate_vertical_motion	ejecuta la simulacion
euler_step	calcula un paso
analytical_solution	calcula referencia
calculate_errors	compara resultados

Ecuacion o logica en papel

```
Euler + formulas analiticas de EX08
```

Comprobacion manual

El primer paso sigue siendo h=5.0, v=49.019, t=0.1. Refactorizar no debe cambiar la fisica.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- funciones
- while
- Euler
- return multiple

Nuevo en este ejercicio:

- separacion de responsabilidades
- funcion que llama a otra
- return True/False
- variables locales
- orquestacion

3. Conceptos Python nuevos

1. Responsabilidad

Sintaxis: `def euler_step(...):`

Cada funcion intenta hacer una tarea concreta. euler_step solo calcula el siguiente estado.

2. Funcion llama a funcion

Sintaxis: `h, v, t = euler_step(...)`

simulate_vertical_motion reutiliza euler_step en lugar de repetir sus ecuaciones.

3. Boolean de validacion

Sintaxis: `return True / return False`

validate_inputs comunica una decision al codigo exterior.

4. Codigo paso a paso

Paso 1. Separar validacion

```
def validate_inputs(h0, v0, t0, g, dt):
    ...
    return True
```

Si una comprobacion falla devuelve False inmediatamente; si todas pasan devuelve True.

Paso 2. Crear euler_step

```
def euler_step(h, v, t, g, dt):
    h_new = h + v * dt
    v_new = v - g * dt
    t_new = t + dt
    return h_new, v_new, t_new
```

La funcion tiene entradas y salidas muy claras y puede probarse sola.

Paso 3. Reutilizar en simulacion

```
while h >= 0:
    h, v, t = euler_step(h, v, t, g, dt)
```

La llamada devuelve tres valores y actualiza el estado.

Paso 4. Separar analitica y errores

```
analytical_solution(...)
calculate_errors(...)
```

La simulacion, la referencia y el calculo de error son responsabilidades distintas.

Paso 5. Orquestar

```
if validate_inputs(h0, v0, t0, g, dt):
    max_h, v_f, t_f, i = simulate_vertical_motion(...)
    ...
```

El bloque principal decide el orden: validar, simular, comparar y mostrar.

5. Flujo de ejecucion

```
validate_inputs
 -> simulate_vertical_motion
    -> euler_step repeatedly
 -> analytical_solution
 -> calculate_errors
 -> print
```

6. Resultado / comprobacion esperada

Los resultados deben coincidir con EX08 para las mismas entradas: ~129.92 m, 10.30 s y -51.04 m/s con dt=0.1.

7. Errores tipicos al empezar

- Duplicar las mismas ecuaciones en varias funciones.
- Usar globales cuando deberian ser parametros.
- Confundir nombre de funcion con llamada.
- Cambiar el orden de retornos.

8. Si solo recuerdas cinco cosas

- Las funciones dividen responsabilidades.
- Una funcion puede llamar a otra.
- True/False puede controlar el flujo.
- return conecta funciones mediante datos.
- Refactorizar no debe cambiar el resultado fisico.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.